Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №7

Дисциплина: Основы программирования на Python

Тема: Принципы SOLID и рефакторинг архитектуры Python-приложения

Вариант 2: Библиотека

Цель работы

Получить практические навыки проектирования и рефакторинга объектно-ориентированных приложений на Python с использованием принципов SOLID. Научиться разделять ответственность классов, выделять архитектурные контракты с помощью Protocol и применять паттерн Dependency Injection (внедрение зависимостей).

Постановка индивидуального задания (Вариант 2. Библиотека)

Разработать систему управления книгами и читателями. Реализовать обязательные предметные операции: выдача книги, возврат, бронирование и уведомление читателя. В архитектуре необходимо применить дополнительную абстракцию NotificationSender с двумя независимыми реализациями: EmailNotificationSender и SmsNotificationSender. Работа выполняется в два этапа: сначала создаётся рабочая версия с намеренными архитектурными проблемами, затем выполняется рефакторинг с разделением ответственности и внедрением зависимостей.

Краткие теоретические сведения

Принципы SOLID представляют собой пять базовых рекомендаций по проектированию программного обеспечения:

- S (Single Responsibility Principle) — принцип единственной ответственности. Класс должен отвечать только за одну логическую задачу.
- O (Open/Closed Principle) — принцип открытости/закрытости. Программные сущности должны быть открыты для расширения, но закрыты для модификации.
- L (Liskov Substitution Principle) — принцип подстановки Барбары Лисков. Объекты производных классов должны корректно заменять объекты базового класса без нарушения логики программы.
- I (Interface Segregation Principle) — принцип разделения интерфейсов. Клиенты не должны зависеть от методов, которые они не используют; большие интерфейсы необходимо разделять на мелкие.
- D (Dependency Inversion Principle) — принцип инверсии зависимостей. Модули высокого уровня не должны зависеть от модулей низкого уровня; оба уровня должны зависеть от абстракций.

Абстрактный класс задаёт контракт через наследование и не позволяет создать экземпляр без реализации абстрактных методов. Protocol из модуля typing описывает структурный контракт: класс соответствует протоколу, если содержит методы с требуемыми сигнатурами, при этом явное наследование не требуется. Dependency Injection — это передача зависимостей объекту извне, обычно через конструктор, вместо их создания внутри класса. Такой подход позволяет подменять реализации при тестировании и расширении системы.

Анализ нарушений SOLID в первоначальной архитектуре

В первоначальной монолитной версии программы (файл monolithic_library.py) были допущены критические архитектурные нарушения.

- Нарушение SRP: класс LibraryManager одновременно управляет хранением данных (коллекциями словарей), содержит бизнес-логику выдачи, напрямую считывает ввод с клавиатуры через input() и отправляет

уведомления. Изменение любого из этих механизмов требует переписывания всего класса.

- Нарушение OCP: способы извещения жёстко завязаны на Email. Если в систему потребуется добавить отправку SMS, разработчику придётся вручную изменять условные конструкции if/elif и логику внутри основного метода обработки данных.
- Нарушение DIP: модуль бизнес-логики LibraryManager напрямую импортирует и создаёт внутри себя экземпляр конкретного низкоуровневого класса EmailSender (self.email_sender = EmailSender()), что создаёт жёсткую связность кода.
- Нарушение ISP: класс-менеджер совмещает функции хранилища, сервиса и интерфейса уведомлений, поэтому любая часть программы вынуждена зависеть от всего набора его методов, включая ненужные.
- Отсутствие абстракций: данные хранятся в виде списков словарей, а операции поиска книги и читателя дублируются в разных ветках меню, из-за чего добавление новой предметной операции затрагивает уже работающий код.

Перечисленные проблемы означают, что любое изменение требований — новый канал уведомлений, другое хранилище или дополнительная операция — приводит к правке центрального класса и повторному тестированию всей программы.

Описание рефакторинга и новой архитектуры

В ходе рефакторинга приложение было разделено на пять независимых слоёв:

- Слой моделей (models.py): содержит чистые структуры данных (dataclass) для сущностей Book, Reader и Loan, а также перечисление LoanStatus.

- Слой интерфейсов (interfaces.py): задаёт архитектурные контракты LibraryRepository и NotificationSender с помощью встроенного механизма typing.Protocol.
- Слой инфраструктуры (infrastructure.py): реализует хранилище в памяти InMemoryLibraryRepository и две полиморфные стратегии отправки уведомлений — EmailNotificationSender и SmsNotificationSender.
- Слой бизнес-логики (services.py): класс LibraryService выполняет операции выдачи, возврата, бронирования и проверки задолженностей, взаимодействуя исключительно с абстрактными протоколами. Зависимости передаются извне через конструктор (Constructor Injection).
- Слой представления (console_ui.py): класс ConsoleUI инкапсулирует в себе всю работу с консольным вводом-выводом, изолируя предметную логику от интерфейса.

Точкой сборки приложения является main.py: он создаёт хранилище, выбирает реализацию отправителя уведомлений и передаёт оба объекта в конструктор сервиса, после чего запускает консольный интерфейс. Замена Email на SMS выполняется изменением одной строки в точке сборки и не затрагивает бизнес-логику.

Сравнение архитектур до и после рефакторинга

Характеристика	До рефакторинга	После рефакторинга
Ответственность классов	Один класс LibraryManager решает все задачи	Каждый класс выполняет одну задачу (SRP)
Зависимости	Прямые жёсткие зависимости между компонентами	Зависимости направлены на абстрактные Protocol (DIP)
Расширяемость	Требует изменения существующего рабочего кода	Новые функции добавляются через создание новых классов (OCP)
Тестируемость	Невозможно протестировать без ручного ввода в консоль	Бизнес-логика легко покрывается Unit-тестами через Mock/Fake объекты
Связность компонентов	Высокая связность (Tight Coupling), компоненты неотделимы	Слабая связанность (Loose Coupling) через внедрение зависимостей

Характеристика	До рефакторинга	После рефакторинга
Использование абстракций	Полностью отсутствует	Применяются интерфейсы-протоколы typing.Protocol

Раздел тестирования бизнес-логики

Стратегия тестирования заключается в полной изоляции бизнес-логики (LibraryService) от внешних инфраструктурных слоёв: консольного интерфейса и реального хранилища. Для достижения этой цели в файле test_library.py реализованы Fake-объекты репозитория и сервиса уведомлений, имитирующие поведение системы без выполнения побочных эффектов. Автоматическое тестирование проводилось с использованием среды pytest.

№	Тест	Проверяемое поведение
1	test_successful_borrow_book	Выдача свободной книги возвращает True, флаг is_borrowed становится True, уведомление отправлено
2	test_borrow_already_borrowed_book	Повторная выдача занятой книги возвращает False
3	test_check_overdue_loans	Просроченная выдача попадает в список задолженностей и сопровождается уведомлением

Все три теста проходят успешно, что подтверждает работоспособность бизнес-логики без участия консоли и реальной базы данных. Интерактивное тестирование проводилось вручную через консольный слой представления console_ui.py для верификации всех четырёх предметных сценариев варианта: выдачи книги с отправкой уведомления, возврата книги, бронирования и проверки должников. Отдельно проверялась замена отправителя уведомлений: при выборе SMS-реализации сообщения выводятся от имени SMS-сервиса, при этом код LibraryService не изменяется.

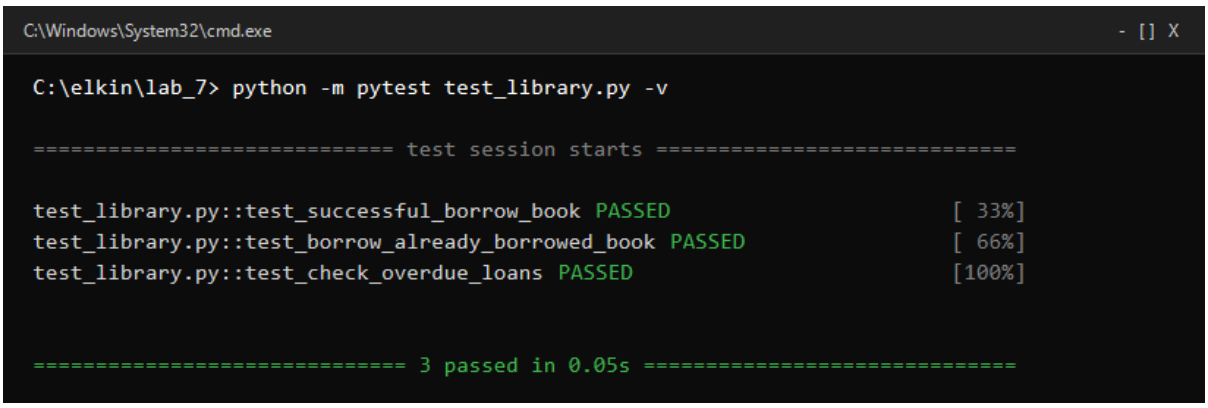


Рисунок 7.1 — Результат успешного прохождения автоматизированных pytest-тестов бизнес-логики

Листинги ключевых фрагментов программы

Листинг 7.1 — Интерфейсы (interfaces.py)

```
from typing import Protocol, List, Optional
from models import Book, Reader, Loan


class LibraryRepository(Protocol):
    def get_book_by_id(self, book_id: int) -> Optional[Book]: ...
    def get_reader_by_id(self, reader_id: int) -> Optional[Reader]: ...
    def add_loan(self, loan: Loan) -> None: ...
    def get_all_loans(self) -> List[Loan]: ...
    def remove_loan(self, book_id: int, reader_id: int) -> bool: ...


class NotificationSender(Protocol):
    def send_notification(self, reader: Reader, message: str) -> None: ...
```

Листинг 7.2 — Бизнес-логика (services.py)

```
from datetime import datetime, timedelta
from typing import List, Tuple
from models import Loan, Book, Reader
from interfaces import LibraryRepository, NotificationSender


class LibraryService:
    def __init__(self, repository: LibraryRepository,
                 notifier: NotificationSender):
        self.repository = repository
        self.notifier = notifier


    def borrow_book(self, book_id: int, reader_id: int) -> bool:
        book = self.repository.get_book_by_id(book_id)
        reader = self.repository.get_reader_by_id(reader_id)
        if not book or book.is_borrowed or not reader:
            return False
        book.is_borrowed = True
        due_date = datetime.now() + timedelta(days=14)
        new_loan = Loan(book_id=book_id, reader_id=reader_id,
                        due_date=due_date)
        self.repository.add_loan(new_loan)
        self.notifier.send_notification(
            reader, f"Вы успешно арендовали книгу '{book.title}'.")
        return True
```

Листинг 7.3 — Реализации хранилища и уведомлений (infrastructure.py)

```
class InMemoryLibraryRepository(LibraryRepository):
    def __init__(self):
        self.books = {...}
        self.readers = {...}
        self.loans = []


    def get_book_by_id(self, book_id: int) -> Optional[Book]:
        return self.books.get(book_id)


    def add_loan(self, loan: Loan) -> None:
        self.loans.append(loan)


class EmailNotificationSender(NotificationSender):
    def send_notification(self, reader: Reader, message: str) -> None:
        print(f"[EMAIL -> {reader.email}] {message}")


class SmsNotificationSender(NotificationSender):
    def send_notification(self, reader: Reader, message: str) -> None:
        print(f"[SMS -> {reader.phone}] {message}")
```

Листинг 7.4 — Тестирование через Fake-объекты (test_library.py)

```
def test_successful_borrow_book():
    repo = FakeRepository()
    notifier = FakeNotifier()
    service = LibraryService(repo, notifier)
    result = service.borrow_book(book_id=1, reader_id=101)
    assert result is True
    assert repo.book.is_borrowed is True


def test_borrow_already_borrowed_book():
    repo = FakeRepository()
    repo.book.is_borrowed = True
    service = LibraryService(repo, FakeNotifier())
    assert service.borrow_book(book_id=1, reader_id=101) is False
```

Вывод

В ходе выполнения лабораторной работы были изучены и применены на практике все пять принципов SOLID. На примере предметной области «Библиотека» осуществлён переход от монолитной структуры к гибкой многослойной архитектуре. Использование механизмов typing.Protocol и паттерна Dependency Injection позволило изолировать ядро бизнес-логики от конкретных способов реализации отправки уведомлений (Email/SMS) и способов хранения данных. Разработанная архитектура является расширяемой, слабосвязанной и полностью готовой к автоматизированному Unit-тестированию.